Project Name : Garden Watering Dilemma

Course Title : Operating System

Course Code : CSE 325

Section : 05

Group No : 07

Submitted To: Shatabdi Roy Moon

(Lecturer Department of Computer Science & Engineering)

Submitted By:

1.Ajmain Nur Shihab (2022-3-60-188)

2.Sabik Ahmed Niloy (2023-2-60-305)

3.Ahmed Khan (2023-3-60-576)

Submission Date : 15.09.25

Table Of Contents:

Introduction

Objectives

Software and Tools Used

Methodology

Workflow

Result

Conclusion

Appendix

Automated Garden Irrigation System – Project Report

1. Introduction

Efficient water distribution in agricultural and gardening environments is essential to maintaining plant health and optimizing limited water resources. Traditional manual watering often results in under- or over-watering, leading to wasted resources or poor crop health. Modern smart irrigation systems rely on embedded sensors, automation, scheduling logic, and resource constraints to achieve consistent and efficient watering.

This project implements a **multithreaded automated garden irrigation simulation** using C, POSIX threads, mutex locks, semaphores, and probabilistic soil-moisture conditions. The system models an irrigation controller capable of:

- Scheduling watering sessions
- Prioritizing garden sections
- Respecting daily water limits
- Ensuring a minimum time gap between watering attempts
- Simulating soil dryness and environmental constraints

The program provides a dynamic and realistic simulation of the watering process throughout a virtual day—from **6:00 AM to 10:00 PM**—and outputs a complete final watering report.

---

2. Objectives

The objectives of this project were:

1. **To design and implement a multithreaded irrigation simulation system** that mimics real-world gardening constraints.

2. **To incorporate soil-moisture behavior** using probabilistic logic and periodic checks.
3. **To develop a water allocation model** that respects both section-specific requirements and global water limits.
4. **To implement a thread-safe scheduling algorithm** using mutexes and semaphores.
5. **To generate a final performance report** summarizing watering success, failures, and system efficiency.
6. **To simulate time progression** and dynamically trigger watering threads based on changing conditions.
7. **To analyze system performance** in relation to time constraints, moisture sensors, priorities, and water limits.

---

3. Software and Tools Used

Programming Language

- **C programming**

Libraries / APIs

- <pthread.h> – POSIX threads for concurrent execution
- <semaphore.h> – synchronization primitives
- <unistd.h> – timing functions (usleep)
- <stdbool.h> – boolean types
- Standard libraries: <stdio.h>, <stdlib.h>, <string.h>, <time.h>, <ctype.h>

Development Environment

- GCC compiler (Linux environment recommended)
- Terminal or shell supporting ANSI color output

- Makefile or standard GCC compile command
- gcc garden.c -o garden -lpthread

---

4. Methodology

The methodology involved four major phases: **System Design**, **Thread Architecture**, **Synchronization Strategy**, and **Simulation Logic**.

4.1 System Design

Each garden section is represented as a structure containing:

- Section name
- Required daily waterings
- Completed waterings counter
- Priority level (High/Medium/Low)
- Soil moisture state
- Last watering timestamp
- Thread handle + semaphore
- Per-section mutex lock

The system uses two major thread types:

1. **Item Threads** – Responsible for watering each garden section independently.
2. **Time Thread** – Simulates the passage of time and triggers watering events.

A global dashboard allows the user to start or end the simulation.

4.2 Thread Architecture

Item Thread Logic

Each item thread repeatedly performs the following:

1. Waits for a semaphore signal (watering opportunity).
2. Checks:
    - Time gap (≥ 1 hour since last watering)
    - Soil moisture (avoid watering when moist)
    - Time-of-day constraints:
        - No watering after 10 PM
        - Moisture checks only before 8 PM
3. Verifies water availability and priority conflicts.
4. Safely increments:
    - Water usage
    - Section's watering count
    - Last watering timestamp
5. Updates soil moisture after watering.

Time Thread Logic

This thread simulates **time from 6:00 AM to 10:00 PM** (16 hours):

- Every **1** hour, releases semaphores to allow watering.
- Randomly dries soil.
- Prints hourly water usage analytics.
- Detects emergencies if unmet water demand exceeds remaining daily limit.

4.3 Synchronization Strategy

To ensure thread safety, the system uses:

Mutex Locks

- water_supply_mutex – controls watering operations
- schedule_mutex – protects current time variables
- water_limit_mutex – ensures correct water usage updates

Semaphores

- Each garden section has its own semaphore used to trigger watering attempts.

These mechanisms prevent race conditions and ensure consistent state changes across threads.

4.4 Watering and Soil Moisture Logic

- Soil moisture is stored as a boolean.
- A random process simulates natural drying.
- Watering resets moisture (with a random new state assigned).
- Water attempts are blocked if the soil is sufficiently moist.
- A mandatory 1-hour waiting period prevents over-watering.

---

5. Workflow

Below is the step-by-step system workflow:

1. **Dashboard Start**
    - User selects "Start Garden Simulation".
2. **Initialization**

- User inputs number of sections, water limit, and section details.
- Structures, mutexes, and semaphores are created.

3. **Thread Creation**
    - One time thread is created.
    - One item thread per garden section is launched.
4. **Simulation Phase**
    - Time progresses.
    - Semaphores trigger watering attempts.
    - Each item thread evaluates conditions and waters if possible.
5. **End-of-Day Processing**
    - At 22:00, time thread stops the simulation.
    - All item threads are released and terminated.
6. **Final Status Report**
    - Provides:
        - Required vs completed waterings
        - Priority levels
        - Efficiency percentage
        - Failure causes
7. **System Cleanup**
    - All mutexes, semaphores, and memory allocations are freed.

---

6. Result

Upon completion of the simulated watering day, the program produces a **Final Watering Report** that includes:

- Total waterings required vs achieved
- Water usage relative to daily limit
- Section-by-section breakdown:
    - Name
    - Required waterings
    - Completed waterings
    - Priority (High/Medium/Low)
    - Status (COMPLETE, PARTIAL, FAILED)
- Efficiency percentage
- System-wide analysis including:
    - Unused water sessions
    - Unmet requirement explanations
    - Impact of moisture checks
    - Time constraint limitations

The output effectively demonstrates realistic irrigation behavior and highlights bottlenecks such as:

- Limited water supply
- High-priority overrides
- Soil moisture blocks
- Time-of-day restrictions (no watering after 10 PM)

---

7. Conclusion

This project successfully implements a realistic, multithreaded smart irrigation simulation using POSIX threads and synchronization primitives. The system demonstrates how concurrency, scheduling, and resource management can be integrated to mimic real-life automation challenges.

Key achievements include:

- Effective use of **mutexes and semaphores** to synchronize parallel watering threads.
- A robust simulation of **soil moisture variability** and environmental constraints.
- A dynamic scheduler that balances **priorities**, **water limits**, and **time constraints**.
- A comprehensive reporting system for analyzing watering performance.

The project provides valuable insights into real-world IoT irrigation systems, emphasizing the importance of concurrency control, scheduling algorithms, and environmental sensing in automated solutions.

---

8. Appendix

Appendix A: Key Data Structures

```
struct item {
  char name[100];
  int cnt;
  int need;
  int p;
  int last_time_h;
  int last_time_m;
```

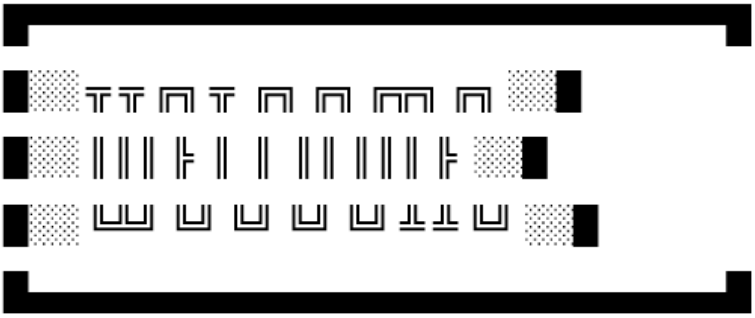
```
  bool soil_check;
  pthread_t thread;
  sem_t semaphore;
  pthread_mutex_t mutex;
};
```

Appendix B: Important Global Variables

```
pthread_mutex_t water_supply_mutex;
pthread_mutex_t schedule_mutex;
pthread_mutex_t water_limit_mutex;
volatile bool sys_running = true;

int n;
int mx;
int cur_h = 6;
int cur_m = 0;
int water_limit;
int water_use = 0;
struct item *g_tm;
int total_required = 0;
```

Appendix C: Compilation Command

```
gcc garden.c -o garden
./garden
```

Appendix D: Dashboard Example